Module 10 • Operations & Deployment

Deployment & Operational Runbook

IRRIGATEWISE Hydrogel-Based Smart Irrigation System Specification

Deployment & Operational Runbook

This guide covers local development, Docker multi-container orchestration, and production deployment for **IRRIGATEWISE**.

---

1. Prerequisites

- **Python:** 3.10 or higher
- **Node.js:** 18 LTS or 20 LTS (with npm)
- **Database:** PostgreSQL 15 or 16
- **Containerization (Optional):** Docker Engine 24+ and Docker Compose v2+

---

2. Option A: Docker Compose Multi-Container Orchestration (Recommended)

The easiest way to run the complete stack (PostgreSQL, FastAPI Backend, Next.js Frontend) is using the root `docker-compose.yml`:

```
# 1. Clone repository
git clone https://github.com/AAnnapoorani/Hydrogel-Based-Smart-Irrigation-System.git
cd "Hydrogel Based Smart Irrigation System"

# 2. Build images and start services in background
docker compose up -d --build

# 3. Verify container health
docker compose ps
```

Exposed Endpoints:

- **Frontend Web Dashboard:** `http://localhost:3000`
- **FastAPI Backend Server:** `http://localhost:8000`
- **Interactive Swagger Docs:** `http://localhost:8000/docs`

- **PostgreSQL Database:** `localhost:5432` (`irrigate_user` / `irrigate_pass`)

---

3. Option B: Local Manual Setup (Without Docker)

Step 1: PostgreSQL Setup

Ensure PostgreSQL is active locally on port 5432:

```
CREATE USER irrigate_user WITH PASSWORD 'irrigate_pass';
CREATE DATABASE irrigatewise OWNER irrigate_user;
```

Step 2: Backend Setup

```
# Navigate to backend
cd backend

# Create virtual environment
python -m venv .venv
source .venv/bin/activate # Linux / macOS
# or: .venv\Scripts\activate # Windows

# Install Python dependencies
pip install -r requirements.txt

# Seed database with baseline tables, user Rajan Kumar, and models
python seed_db.py

# Launch FastAPI ASGI server
uvicorn main:app --host 0.0.0.0 --port 8000 --reload
```

Step 3: Frontend Setup

```
# Navigate to frontend in a new terminal
cd frontend

# Install dependencies
npm install

# Run development server
npm run dev

# Or build and launch production standalone server
npm run build
npm start
```

---

4. Running the ESP32 IoT Edge Simulator

To emulate physical ESP32 edge telemetry and populate live sensor charts:

```
# Run 48 consecutive hourly readings at 1-second intervals
python backend/simulator/esp32_simulator.py --cycles 48 --interval 1.0

# Run continuous infinite simulation streaming to backend
python backend/simulator/esp32_simulator.py --url http://localhost:8000/api/v1/sensors/readings
```

---

5. Automated Verification & Test Suite

Run the full pytest suite to verify ML models, decision engines, anomaly detection, and API routes:

```
python -m pytest tests/ -v
```

Expected output: `18 passed in < 2.0s`.